RV College of Engineering®

UNIT 1
Introduction to Data Structures

By: Prof. Rajesh R M
Assistant Professor
Dept. of AI and ML
RVCE, Bangalore

Go, change the world®

Outline

- Data and Information
- Data Structure
- Data Type
- Abstract Data Type
- Classification of Data Structure

Data and Information

Data is a plural of datum, which is originally a Latin noun meaning —something given

The Oxford dictionary meaning of data is:

i) Facts or statistics used for reference or analysis.

ii) The quantities, characters or symbols operated by a computer

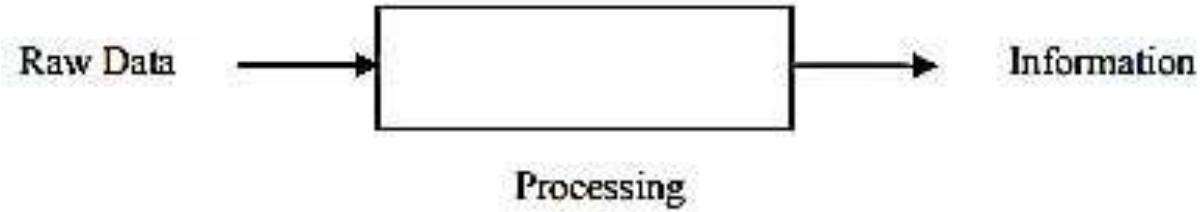
Definition:

Data is the basic entity or fact that is used in a calculation or manipulation process.

- There are two types of data, such as **numerical** and **alphanumerical** data.
- Data may be a single value or a set of values and it is to be organized in a particular fashion.

Information

- Data are raw facts without context, whereas Information is data with context.
- Data are an abstraction of Information in a problem-solving system.



Representation of Data

Almost all the high-level languages, e.g. FORTRAN, COBOL, C, Pascal, C++, Java, etc. allow data to be represented in computer memory of mainly two types:

i) Numerical Data

ii) Alphanumerical Data

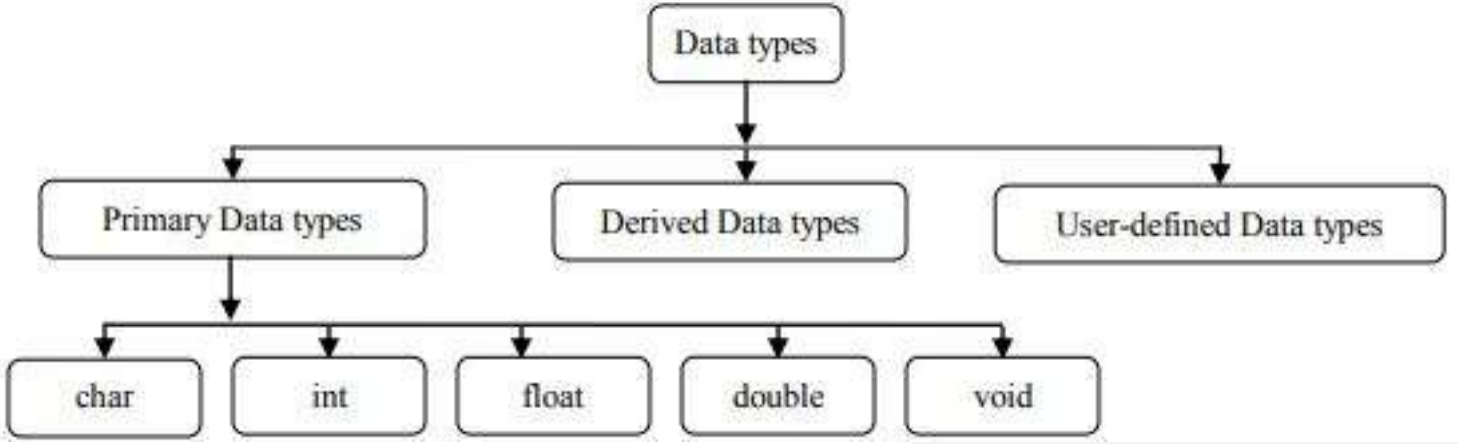
Most of the high-level languages classified numerical data into two types:

i) Integer

ii) Floating-point

Data Type

A data type refers to the type of data that variables hold.



Data Structure

Data Structure is the representation of the logical relationship between individual elements of data.

The purpose of studying data structure is to learn how to organize data into **memory** so that it can be accessed **quickly**, **conveniently** and **Efficiently**.

Data Structure refers to the study of data and representation of data objects within the program; i.e., the implementation of structured relationships among different data objects.

Example: lists, stacks, queues, heaps, search trees, hash tables, etc

Data Structure

Data Structure can be used for the following purpose:

i) **Organizing the data** – How data items are organized in the main memory?

ii) **Accessing methods** – How data items can be accessed?

iii) **Specifying the degree of associativity** – How data items are interrelated?

iv) **Processing alternatives for data** – How many different ways are there in which these data items can be processed?

Classifications of Data Structure

Data Structure can be classified into two categories:

i) Primitive data structure

ii) Non-Primitive data structure

Primitive Data Structure

The primitive data structures are defined that can be **manipulated** or **operated** by the **machine instruction**. There are numerous types of data structures, generally built upon simpler primitive data types, called Primitive data structures, which are **represented in computer memory**.

Example: Integer, floating point, characters, pointer, boolean, etc.

Classifications of Data Structure

Data Structure can be classified into two categories:

i) Primitive data structure

ii) Non-Primitive data structure

Non-primitive Data Structure

The non-primitive data structures are a data structure that cannot be **manipulated** or **operated** directly by the **machine instructions**. These are more **sophisticated** data structures. Non-primitive data structures are **derived** from **the primitive data structure**. These are a group of homogeneous or heterogeneous data items.

Example: Arrays, structure, stack, queues, linked lists etc.

The Non-primitive data structures are classified into two categories:

i) Linear data structure

ii) Non-linear data structure

Linear Data Structures

- The data structure is linear if every data item is related to its previous and next data items (e.g., array, linked list).
- In the linear data structure, data items are arranged in memory in a linear sequence and data items are accessed linearly.
- The traversing of the linear data structure is exactly once.

Linear data structures are two types:

i) Sequential :- Sequential Data Structures are based on arrays where objects are stored in a sequence of consecutive memory locations. **Example**: Arrays, Stacks, Queues

ii) Linked :- Linked Data Structure is a data structure, which consists of a set of nodes linked together and organized with links. Example: Linked Lists, Linked Stacks, Linked Queues

The Non-primitive data structures are classified into two categories:

i) Linear data structure

ii) Non-linear data structure

Non-linear Data Structures

- A data structure is non-linear if every data item attaches to many other data items in specific ways to reflect relationships (e.g., tree).
- In non-linear data structures, the data elements are not in sequence, i.e., insertion and deletion are not possible in a linear manner.
- The traversing of the non-linear data structure is always more than one.

Example: Graphs, Trees

Static Data Structure

The static data structure is a kind of data structure, in which once memory space is allocated it cannot extend, i.e. the memory allocation for the data structure takes place at compile-time that cannot be changed afterwards.

Example: Array

Dynamic Data Structure

Dynamic Data Structure is another kind of data structure, which can be extended or shrink during the execution, i.e., the memory allocation as well as memory de-allocation for the data structure takes place at run-time and allocates memory as required amount at any time.

Example: linked list, stack, queue, tree

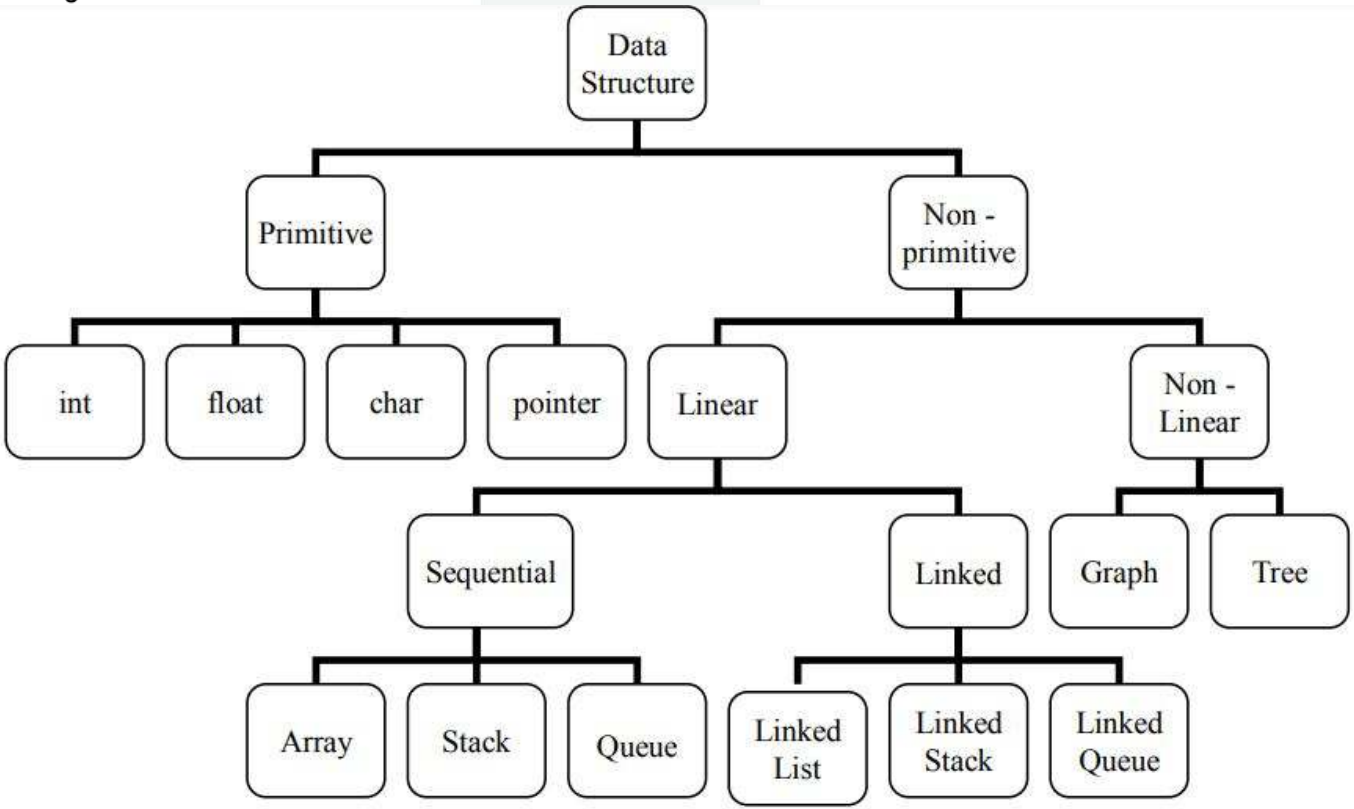


Figure: Classification of Data Structures

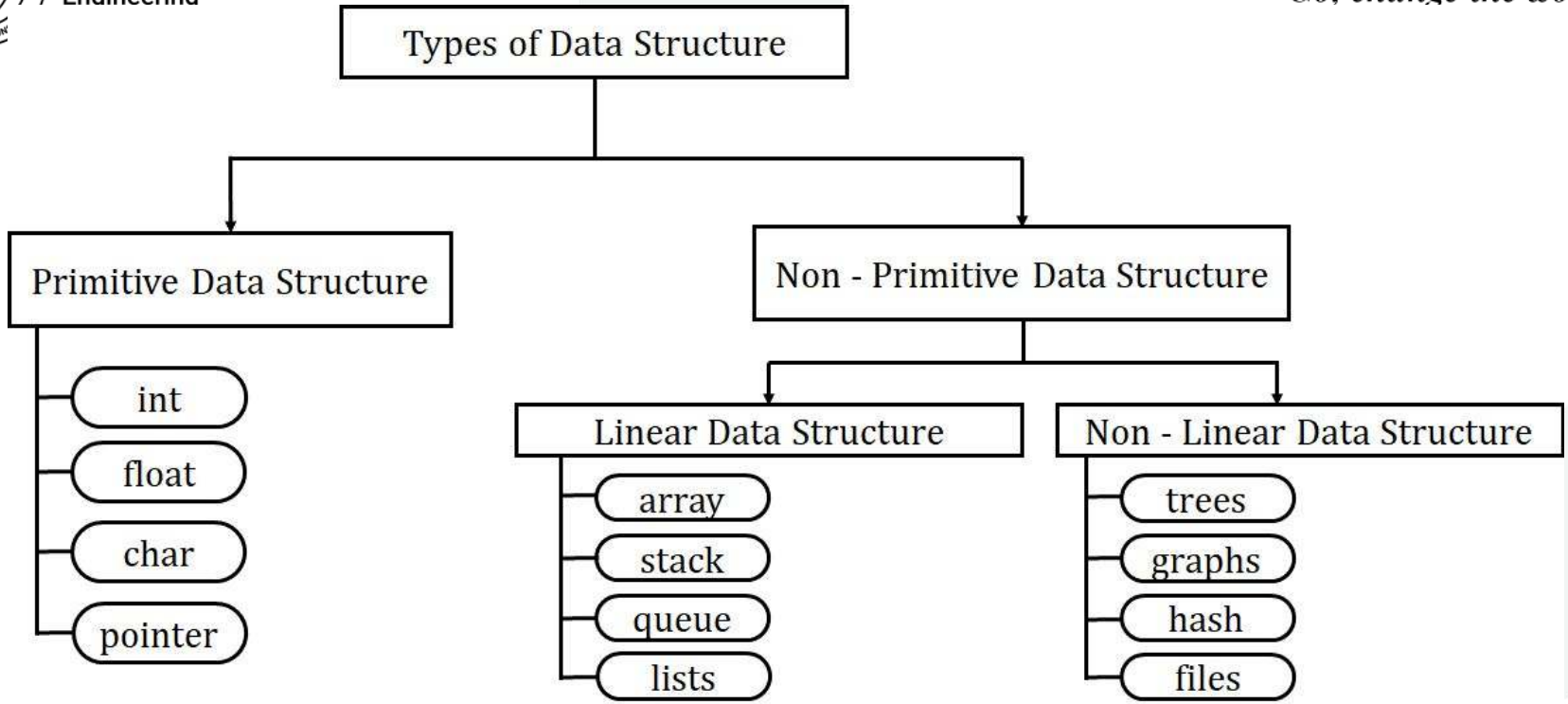


Figure: Classification of Data Structures

Applications of Data Structure

- Compiler Design
- Operating System
- Database Management System
- Statistical Analysis Package
- Numerical Analysis
- Graphics
- Artificial Intelligence
- Simulation

Operations on Non – Primitive Data Structure

Creating :- The process of environment creation to store data.

Inserting :- The process of adding the elements to the data structure at any location.

Deleting: - The process of removing an element from the data structure is called Deletion. We can delete an element from the data structure at any random location.

Searching: - The process of finding the location of an element within the data structure.

Sorting: - The process of arranging the data structure in a specific order is known as Sorting.

Merging: - The process of combining two lists in to single list.

Traversing: - Traversing the data structure means visiting each element of the data structure in order to perform some specific operation like searching or sorting.

Abstract Data Type

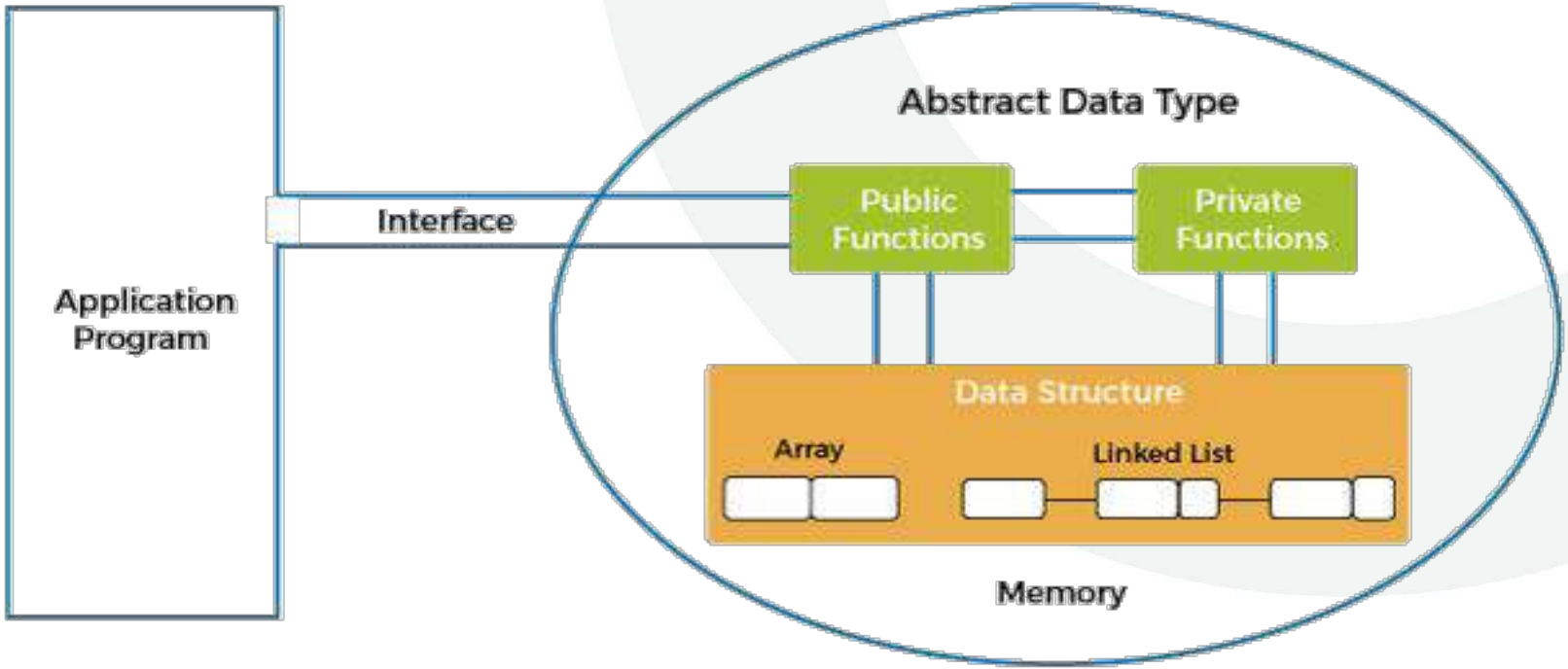
- An Abstract Data Type (ADT) describes the data objects, which constitute the data structure and the fundamental operations supported on them.
- An ADT promotes data abstraction and focuses on what a data structure does, rather than how it implements (does).

✔ An ADT is a conceptual model of information structure.

✔ An ADT specifies the components, their structuring relationships and a list of operations that are allowed to perform.

✔ Specification involves the —what the operations, not the —how

Abstract Data Type



Overview of Different Data Structures

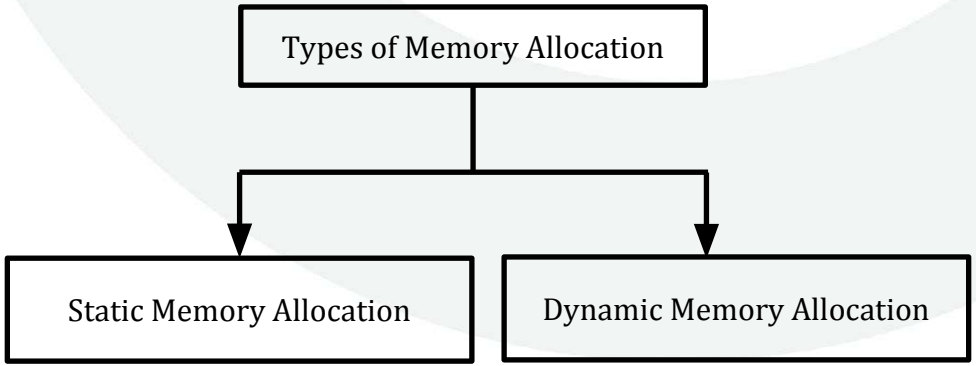
- Array
- Stack
- Queue
- Linked List
- Tree
- Graph

Memory Allocation

Memory allocation is the process of reserving a partial or complete portion of computer memory for the execution of programs.

Types of Memory Allocation

- Static Memory Allocation
- Dynamic Memory Allocation



Static Memory Allocation

- "The process of allocating memory space during compile time".
- Once the memory is allocated,
  - Memory space cannot be extended to accommodate more data during runtime.
  - Memory space cannot be reduced to accommodate less data during run time.

Advantage of Static Memory allocation

- Execution of program is faster.
- Applicable for fixed size data.
- Requires less memory to execute programs.

Disadvantage of Static Memory allocation

- Static memory is fixed, explicitly user cannot alter the memory during execution.
- Leads to under utilization if more space is allocated.
- Leads to overflow if less memory is allocated.
- You can use static memory only if you know the data is fixed.

- **Dynamic Memory Allocation**
  - The process of allocating memory space during compile time.
  - Once the memory is allocated,
    - Memory space can be extended to accommodate more data during runtime.
    - Memory space cannot be reduced to accommodate less data during run time.
    - Memory space is allocated in **heap area.**

Advantage of Dynamic Memory allocation

- Based on the requirement user can choose the memory
- Efficient utilization of memory.
- User can choose Dynamic Memory allocation when not aware of data size.
- When the memory not in use we can release memory at runtime.

Disadvantage of Static Memory allocation

- Require additional functions to manage memory.
- Execution is slow compared to static memory.
- Pointer is dangerous and difficult to handle.
- User should deallocate memory allocated at runtime.

Static Memory Allocation		Dynamic Memory Allocation	
1	Memory Allocation done at compile time	1	Memory Allocation done at run time
2	Size of the memory is fixed	2	Size of the memory is varies depends on the user requirement
3	Memory space cannot be extended to accommodate more data during runtime.	3	Memory space can be extended to accommodate more data during runtime.
4	Memory space cannot be reduced to accommodate less data during run time.	4	Memory space cannot be reduced to accommodate less data during run time.
5	Memory management is easy.	5	Memory management requires additional functions so its difficult to manage.
6	Memory de allocation is done by the compiler.	6	User explicitly need to de allocate memory using built-in functions
7	Execution of program is fast because memory already allocated.	7	Execution of program is slow because memory allocation done at runtime.
8	Memory is allocated at stack area or in data area.	8	Memory is allocated in heap area.

Memory Management Functions

- malloc
- calloc
- realloc
- free

1. malloc (size)

- This function allows the program to allocate block of memory as and when required.
- Block size should be specified in the function parameter.
- It allocates memory of size **bytes**

Syntax

```
ptr = (data_type *) malloc (size);
```

Where **ptr** is a pointer variable.

data_type can be any of the basic data type or user defined data type.

size is the number of bytes required.

Example

```
int  * ptr;
ptr=( int *) malloc (sizeof(int)* 10)
```

Note:

- On successful allocation of memory malloc function returns the **"address of first byte of allocated memory".**
- If memory not available malloc return **"overflow of memory"** and returns **NULL.**

2. calloc (n, size)

- This function is used to allocate **contiguous allocation of multiple block**.
- Mainly used to allocate memory for **array**. Allocates multiple blocks of memory each block with **same size**
- Number of blocks is determined by the first parameter **n**.

Syntax

```
ptr = (data_type *) calloc (n, size);
```

Where **ptr** is a pointer variable.

data_type can be any of the basic data type or user defined data type.

n is the number of blocks to be allocated.

size is the number of bytes in each block.

Example

```
int  * ptr;
ptr=( int *) calloc (5, sizeof(int));
```

Note:

- On successful allocation of memory malloc function returns the **"address of first byte of allocated memory"**.

	Malloc( )		Calloc( )
1	Syntax of malloc( ) is `ptr=(data_type*)malloc(size);`	1	Syntax of calloc( ) is `ptr=(data_type*)calloc(n, size);`
2	Allocate a block of memory of size **bytes**	2	Allocates multiple blocks of memory each block with **same size**
3	Allocated space will not be initiated.	3	Each byte of allocated space in initialized to **Zero**
4	Since no initialization is takes place time efficiency is high compared to calloc	4	Since initialization is takes place time efficiency is low compared to malloc
5	If continuous memory location is not available in memory, allocation done at different and random location.	5	It allocates memory only the required space is available in memory otherwise it returns NULL.

6	Initialization of memory can be done by sing the following statement
	`ptr=malloc(sizeof(int) * n);`
	`memset(ptr, 0, sizeof(int) * n);`
		6	Implicitly the function do the memory initialization
			`ptr=(int*)calloc(n, sizeof(int));`

3. realloc (ptr, size)

- This function is used to re-allocate memory.
- The allocated memory can be reduced or it can be increased based on the requirement.
- When memory is extended or reduced pointer value will no be changed.
- If memory cannot be extended, it allocated a completely new block and copies the content of existing memory and deletes old memory.

Syntax

```
ptr = (data_type *) realloc (ptr, size);
```

Where **ptr** is a previously allocated memory address.

data_type can be any of the basic data type or user defined data type.

n is the number of blocks to be allocated.

size is the number of bytes in each block.

Example

```
ptr=( int*) calloc (ptr, 4*sizeof(int));
```

Note:

- On successful allocation of memory malloc function returns the **"address of first byte of allocated memory".**
- If memory not available malloc return **"overflow of memory"** and returns **NULL.**

4. free(ptr)

This function is used to de-allocate block of memory which is allocated by calloc( ), malloc( ), and realloc( ) functions & the pointer should be initialized to **NULL**

Syntax

```
free(ptr);
ptr=NULL;
```

Linked Lists

Definition: A linked list is a linear ordered collection of finite homogeneous data elements called node, where the linear order is maintained by means of links or pointers.

A linked list allocates space for each element separately in its own block of memory called a "linked list element" or "node". The list structure is created by the use of pointers to connect all its nodes together like the links in a chain.

Linked Lists

Property	Array	Linked List
Storage	Storage of the array must be available contiguously.	Storage need not be contiguous.
Memory utilization	The size of the array needs to be predicted beforehand because memory allocation is done in advance.	Memory of linked list is not pre-allocated, memory is allocated whenever it is required.
Memory utilization	Not necessary for storing addresses of any element.	Extra memory space is necessary for storing addresses of the next node
Change of size	The array size is fixed, extend or shrink not possible during the execution of a program	Linked list may extend or shrink during the execution of a program
Insertion/ deletion	Insertion/deletion operations are slow, half of the elements are required to move on an average	Insertion/deletion operations are performed very fast, in a constant amount of time
Searching	Linear searching, binary searching, interpolation searching are possible	Binary searching, interpolation searching not possible, only linear searching is possible
Access element	Fast access to any element in a constant amount of time.	To access any element in a linked list, traversing is required.
Joining/ splitting	Joining and splitting of the two arrays is difficult.	Joining and splitting of two linked list is very easy.

Advantages of linked list

Linked lists have many advantages. Some of the very important advantages are:

i) Linked list are dynamic data structures. That is, they can extend or shrink during the execution of a program.

ii) Storage need not be contiguous.

iii) Efficient memory utilization. Here memory is not pre-allocated. Memory is allocated whenever it is required.

iv) Insertion or deletion is easy and efficient, may be done very fast, in a constant amount of times, independent of the size of the list.

v) The joining of two linked lists can be done by assigning pointer of the second linked list in the last node of the first linked list.

Types of Linked list

There are different types of linked list. We can put linked lists into the following four types:

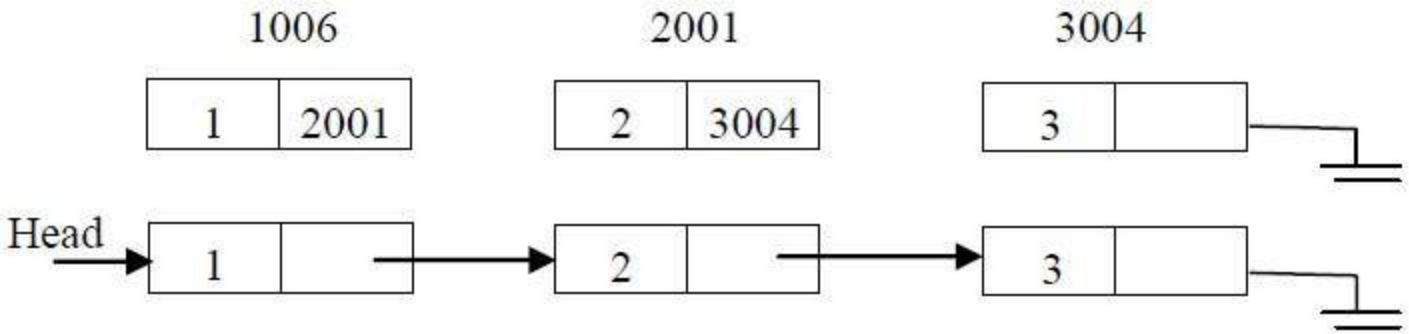
i) Singly linked list
ii) Circular Linked list
iii) Doubly Linked list
iv) Circular Doubly linked list

Singly linked list

An element in a linked list is specially termed as a node. In a singly linked list, each node consists of two fields:

i) DATA field that contains the actual information of the element.

ii) LINK field, contains the address of the next node in the list.



Singly linked list

Operation	Description
Createlist	This operation creates a linked list.
Traverse	This operation traverse/visit all the elements of the linked list exactly once
Insertion	This operation inserts an element to the linked list
Deletion	This operation removes an element from the linked list
Searching	This operation performs linear searching for a key value in the linked list
Reverse	This operation performs the reverse of the linked list
Merging	This operation performs merging of two linked lists in a single linked list

Singly linked list

A linked list is constructed by the nodes. These nodes are allocated in the heap. Each node contains a single data element and a pointer to the next node in the list.

```
struct node
{
    int DATA;
    struct node *LINK;
};
```

The Node structure of a linked list is shown here. Where DATA contains value and LINK holds the address of the next node.

Algorithm to create a singly Linked List

```
Algorithm: CREATE (HEAD, ITEM)
1. [Create NEW node]
   a) Allocate memory for NEW node.
   b) IF NEW = NULL then Print: "Memory not Available" and Return
   c) Set NEW→DATA = ITEM
   d) Set NEW→LINK = NULL
2. [Whether List is empty, head is the content of HEADER]
   If HEAD = NULL then Set HEAD = NEW
3. Else
   a) Set Temp = HEAD
   b) While Temp→LINK ≠ NULL do
         Set Temp = Temp→LINK
      [End of while]
   c) Set Temp→LINK = NEW
   [End of IF]
4. Return
```

Algorithm to traverse a Linked List

```
Algorithm: TRAVERSE(HEAD)
1. If HEAD = NULL then
      i) Print: "The linked list is empty"
      ii) Return
2. Temp = HEAD
3. Repeat while Temp ≠ NULL
      i) Print: Temp→DATA
      ii) Set Temp = Temp→LINK
   [End of Loop]
4. Return
```

Insertion in Singly Linked List

Insertion operation in a singly linked list can be done in different ways using position.

- Insertion at beginning.
- Insertion in the middle.
- Insertion at end.

Algorithm to insert a node at the beginning

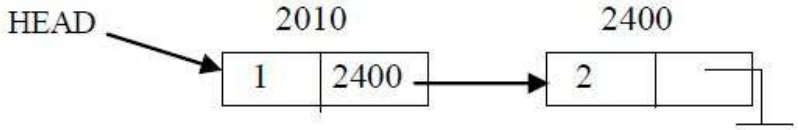
```
Algorithm:ADD_BEG (HEAD, ITEM)
1. [Create the new node]
   a) Allocate memory for NEW node.
   b) IF NEW = NULL then Print: "Memory not Available" and Return
   c) Set NEW→DATA = ITEM
   d) Set NEW→LINK = HEAD
2. [Make the HEADER to point to the NEW node]
   Set HEAD = NEW
3. Return
```

Linked Lists

Algorithm to insert a node at the beginning

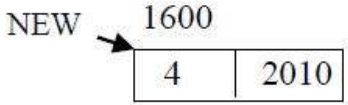
```
Algorithm:ADD_BEG (HEAD, ITEM)
1. [Create the new node]
   a) Allocate memory for NEW node.
   b) IF NEW = NULL then Print: "Memory not Available" and Return
   c) Set NEW→DATA = ITEM
   d) Set NEW→LINK = HEAD
2. [Make the HEADER to point to the NEW node]
   Set HEAD = NEW
3. Return
```

At first HEAD points to the first node of the list containing two nodes.



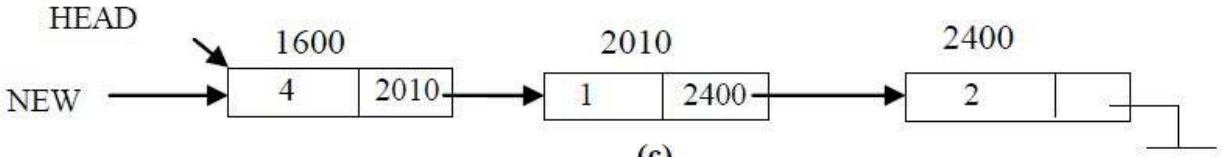
(a)

A new node is created and pointed by pointer NEW. LINK field of new node contains the address of head node.



(b)

Now, HEAD pointer points to the new node



(c)

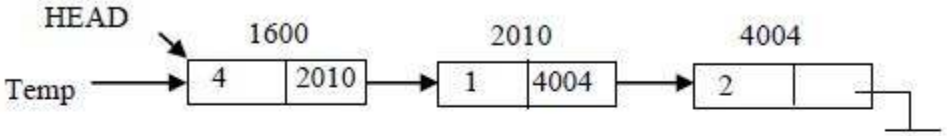
Insertion of a node before a specified node of a singly linked list

This algorithm creates a node and inserts it before the node pointer P. The HEAD is a pointer that points to the first node of the linked list and ITEM is the value of the new node. NEW is a pointer that holds the address of the new node. Temp and PTemp are two temporary pointers to traverse the list.

Algorithm to insert a node before a given node pointer

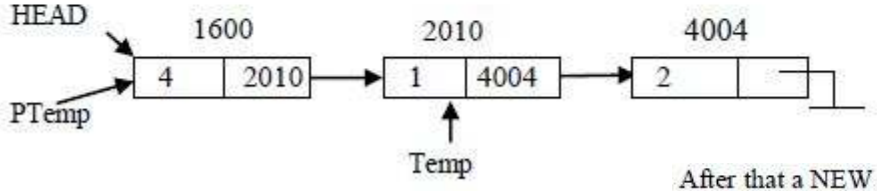
```
Algorithm: ADD_BEFORE (HEAD, ITEM, P)
1. Set Temp = HEAD   [to make temp to point to the first node]
2. Repeat step 3 while Temp ≠ P
3. a) Set PTemp = Temp
   b) Set Temp = Temp→LINK
   c) If Temp = NULL then
      i) Print: "Not Found"
      ii) Return
   [End of loop]
4. [Create the new node]
   a) Allocate memory for NEW node
   b) IF NEW = NULL then Print: "Memory not Available" and Return
   c) Set NEW→DATA = ITEM
   d) Set NEW→LINK = Temp
   e) Set PTemp→LINK = NEW
5. Return
```

At first, Temp is assigned to HEAD.



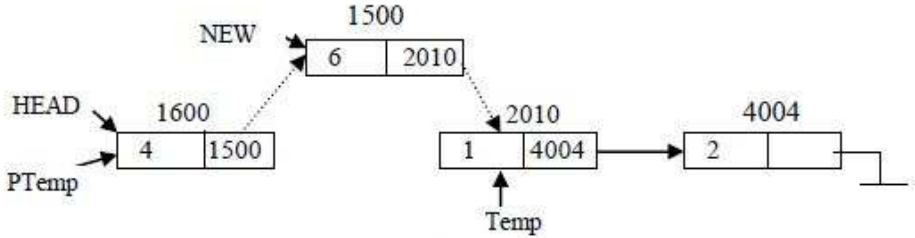
(a)

In the next step, Temp is moved to the next node and PTemp is assigned to the address of the previous node



(b)

After that a NEW node is created and linked with previous and next node.



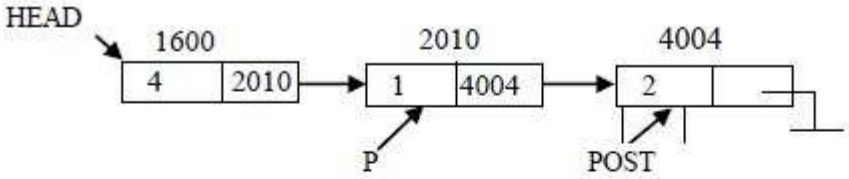
(c)

Insertion of a node after a node of a singly linked list

In the following algorithm, it is described how to insert a node after a specific node pointer P in a linked list. The HEAD is a pointer which points to the first node of the linked list and ITEM is the value of the new node. NEW is a pointer which holds the address of the new node. POST is a temporary pointer that points to the next node of P.

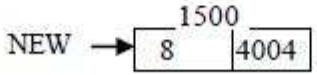
Algorithm to insert a node after a given node pointer

```
Algorithm: ADD_AFTER (HEAD, ITEM, P)
1. Set POST = P→LINK
2. [Create the new node]
   a) Allocate memory for NEW node
   b) IF NEW = NULL then Print: "Memory not Available" and Return
   c) Set NEW→DATA = ITEM
   d) Set NEW→LINK = POST
   e) Set P→LINK = NEW
3. Return
```



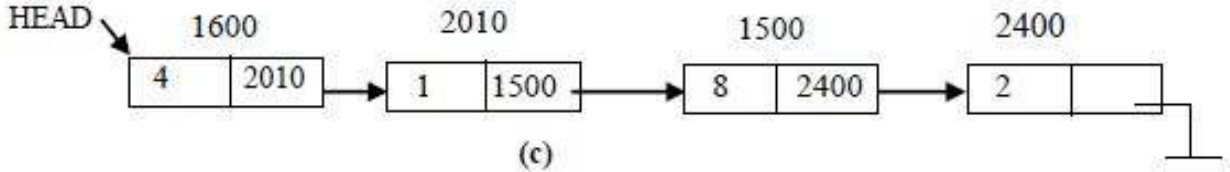
(a)

After that a NEW node is created, ITEM is placed in DATA field and POST is assigned to the LINK field which is shown in figure 5.6b.



(b)

At last, NEW node is assigned to LINK of node P



(c)

Insertion of a node at the end of a singly linked list

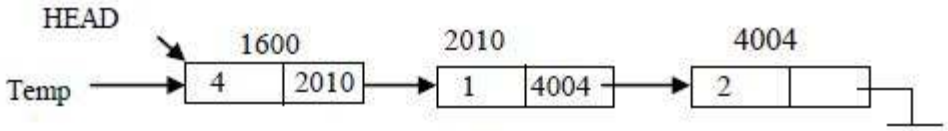
The following algorithm describes how the new node is inserted at the end of a singly linked list. The HEAD is a pointer which points to the first node of the linked list and NEW is a pointer which holds the address of the new node. ITEM is the value of the new node. Temp holds the address of header node.

Insertion of a node at the end of a singly linked list

Algorithm to insert a node at end

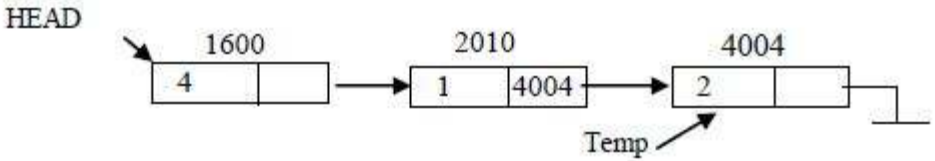
```
Algorithm: ADD_END (HEAD, ITEM)
1. [Create the new node]
   a) Allocate memory for NEW node
   b) IF NEW = NULL then Print: "Memory not Available"  and Return
   c) Set NEW→DATA = ITEM
   d) Set NEW→LINK = NULL
2. Set Temp = HEAD   [to make Temp to point to the first node]
3. Repeat while Temp→LINK ≠ NULL
      Set Temp = Temp→LINK
   [End of loop]
4. Set Temp→LINK = NEW
5. Return
```

At first Temp is assigned to the address of the HEAD node of the linked list

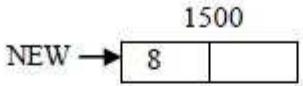


(a)

Now, Temp moves to the end of the linked list.

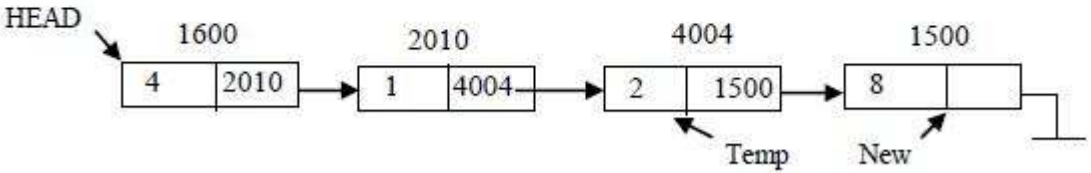


A NEW node is created ITEM is placed in DATA field and NULL is assigned to LINK field.



(c)

At last, NEW is assigned to the LINK field of the last node of the linked list.



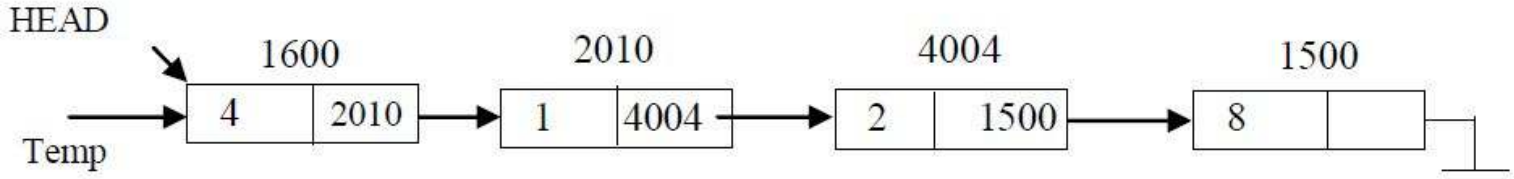
Deletion from a singly linked list

Deletion operation in a singly linked list can be done in different ways using position.

- Deletion from beginning.
- Deletion in the middle.
- Deletion from end.

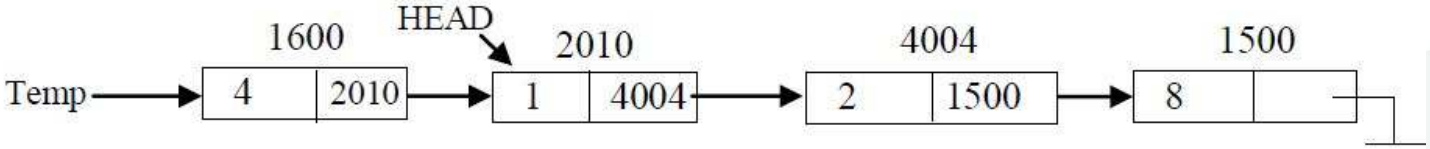
Algorithm to delete a node from the beginning

```
Algorithm: DELETE_BEG (HEAD, ITEM)
1. [Check for empty list]
   IF HEAD = NULL then Print: "The linked list is empty" and Return
2. Set Temp = HEAD      [To make Temp to point the first node]
3. Set HEAD = Temp→LINK
4. Set ITEM = Temp→DATA
5. Set Temp→LINK = NULL
6. Deallocate memory for Temp Node
7. Return
```



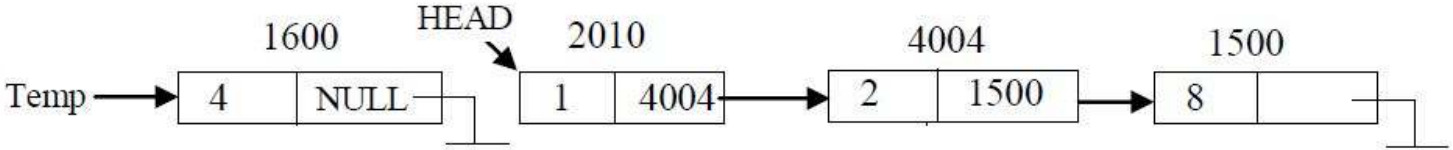
(a)

In the next step HEAD moves to the next node.



(b)

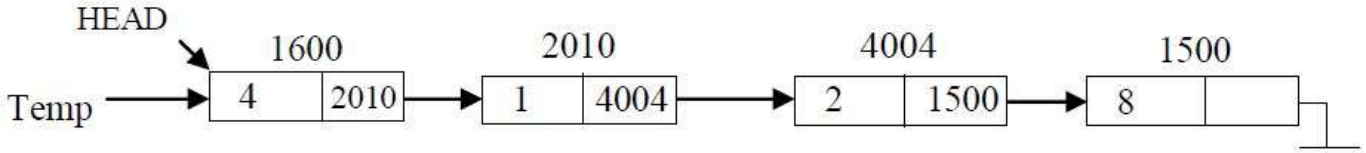
Finally, the first node of the linked list is deleted which is pointed by Temp.



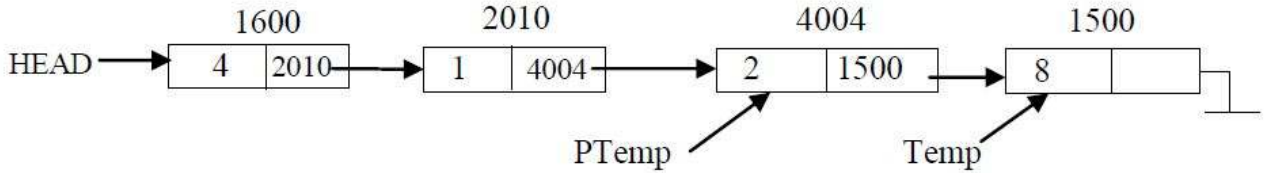
(c)

Algorithm to delete a node from the end

```
Algorithm: DELETE_END (HEAD, ITEM)
1. [Check for empty list]
   IF HEAD = NULL then Print: "The linked list is empty" and Return
2. Set Temp = HEAD
3. Repeat while Temp→LINK ≠ NULL
      a) PTemp = Temp
      b) Set Temp = Temp→LINK
   [End of loop]
4. Set ITEM = Temp→DATA
5. Set PTemp→LINK = NULL
6. Deallocate memory for Temp Node
7. Return
```

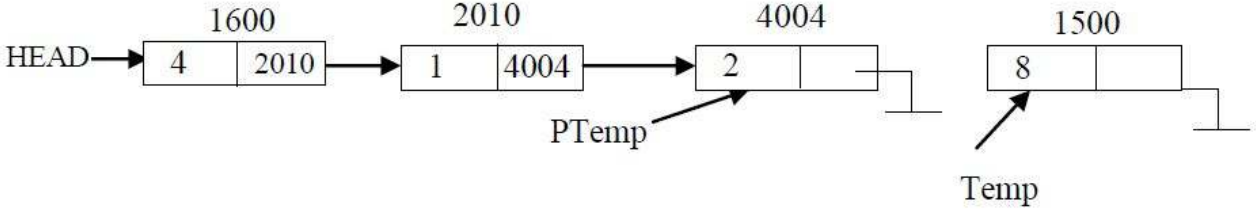


(a)



(b)

At last, PTemp LINK field is assigned to NULL and Temp is deleted.



(c)

Algorithm to delete a node after a given node pointer

```
Algorithm: DELETE_AFTER (ITEM, P)
1. Set Temp = P→LINK
2. Set P→LINK = Temp→LINK
3. Set ITEM = Temp→DATA
4. Set Temp→LINK = NULL
5. Deallocate memory for Temp Node
6. Return
```

Circular Linked List

A circular linked list is the variation of singly linked list in which the last node points to the first node of the list. The circular linked lists have neither a beginning nor an end.

In a circular linked list, there are two methods to know if a node is the first node or not.
i) Either an external pointer, list, points the first node or

ii) A header node is placed as the first node of the circular list.

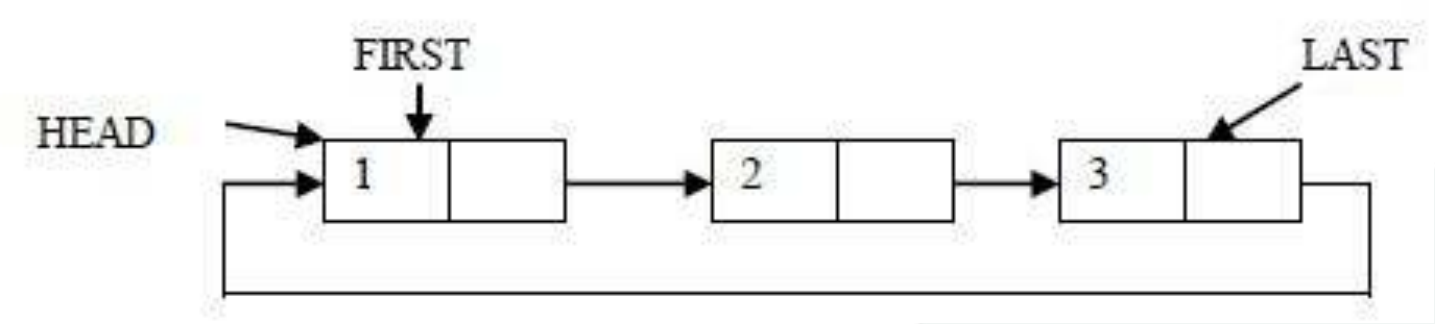
The header node can be separated from the others by either having a sentinel value as the info part or having a dedicated flag variable to specify if the node is a header node or not.

A linked list can be implemented either in two ways:
i) A single pointer is used to point to the header node of the linked list and the last node of the list points to the header node of the list.
ii) Two different pointers can be used to point to the first and last node of the circular linked list respectively.

Circular Linked Lists

A circular linked list can easily be formed from a linear linked list where the last node of linear linked list points to the first node of the linked list.

The circular linked list can be represented by three nodes as follows



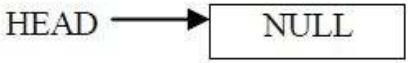
Circular Linked Lists

Operation	Description
Createlist	This operation creates a linked list.
Traverse	This operation traverse/visit all the elements of the linked list exactly once.
Insertion	This operation inserts an element to the linked list.
Deletion	This operation removes an element from the linked list.
Searching	This operation performs linear searching for a key value in the linked list.
Reverse	This operation performs the reverse of the linked list.
Merging	This operation performs merging of two linked lists in a single linked list.

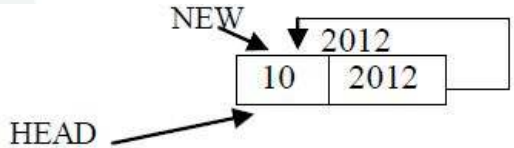
Circular Linked Lists


```
Algorithm: CIRCULAR_ADD (HEAD, ITEM)
1. [Create the new node]
   a) Allocate memory for NEW node
   b) IF NEW = NULL then Print: "Memory not Available" and Return
   c) Set NEW→DATA = ITEM
   d) Set NEW→LINK = NEW
2. [Check whether List is empty or not]
   If HEAD = NULL then
      Set HEAD = NEW
3. Else
   a) Set Temp = HEAD
   b) Repeat while Temp→LINK ≠ HEAD do
         Set Temp = Temp→LINK
      [End of loop]
c) Set Temp→LINK = NEW
4. [Make a new node to point to the first node]
   Set NEW→LINK=HEAD
5. Return
```

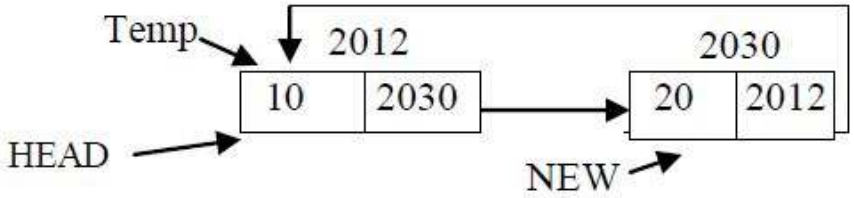
Circular Linked Lists



(a)



(b)



(c)

Circular Linked Lists

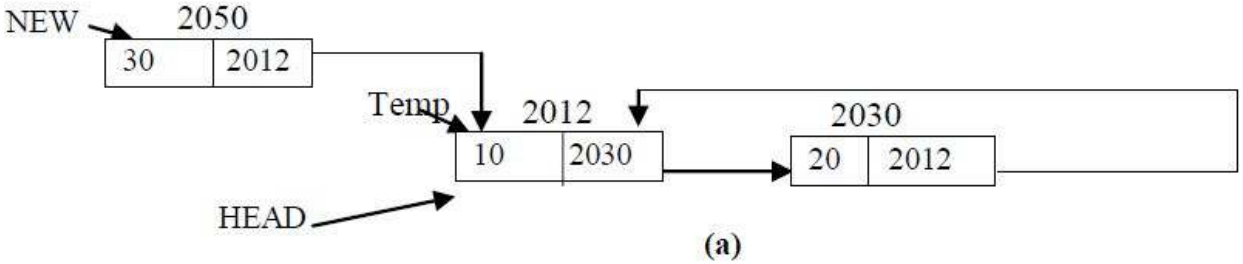
Algorithm to display a Circular Linked List

```
ALGORITHM: DISPLAY (HEAD)
1. [Make Temp to point to the first node]
   Set Temp = HEAD
2. [Check for empty list]
   If HEAD = NULL then
      Print: "The linked list is empty" and Return
3. Print: Temp→DATA
      Set Temp = Temp→LINK
4. Repeat While Temp! = HEAD
      Print: Temp→DATA
      Set Temp = Temp→LINK
5. Return
```

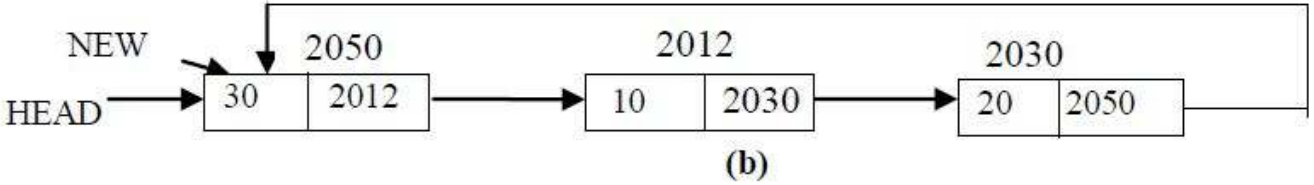
Algorithm to insert a node at the beginning

```
Algorithm: CIRADD_BEG (HEAD, ITEM)
1. [Create the new node]
a) Allocate memory for NEW node.
b) IF NEW = NULL then Print: "Memory not Available"  and Return
c) Set NEW→DATA= ITEM
d) Set NEW→LINK= HEAD
2. a) Set Temp=HEAD
   b) Repeat while Temp→LINK ≠ HEAD do
         Set Temp = Temp→LINK
      [End of loop]
3.  [Make FIRST to point to the new node]
      Set HEAD = NEW          // to make new node the first node
      Set Temp→LINK=HEAD
4. Return
```

Circular Linked Lists



(a)

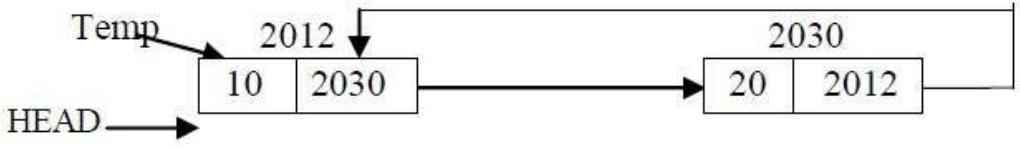


(b)

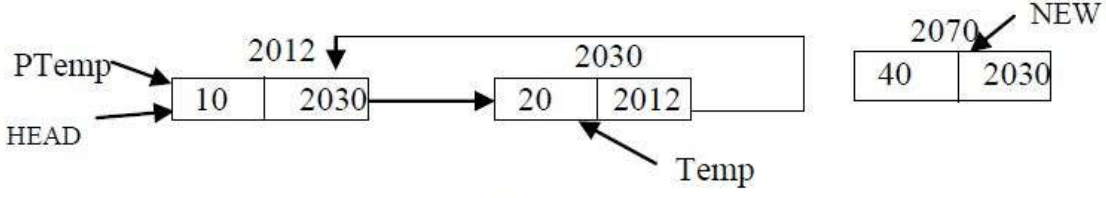
Algorithm to insert a node after given node pointer

```
Algorithm: CIRADD_AFTER (HEAD, ITEM, LOC)
1. [Create the new node]
    a) Allocate memory for new node.
    b) IF NEW = NULL then Print: "Memory not Available" and Return
    b) Set NEW→DATA = ITEM
    c) Set NEW→LINK= Temp
2. [Make Temp point to the first node]
   Set Temp = HEAD
3. Repeat step 3 for i=1 to LOC
3.  a) Set  PTemp = Temp and   Temp = Temp→LINK
    b) If Temp = HEAD then
       Print: "There are less than loc+1 elements" and Return
4. Set PTemp→LINK = NEW
5. Return
```

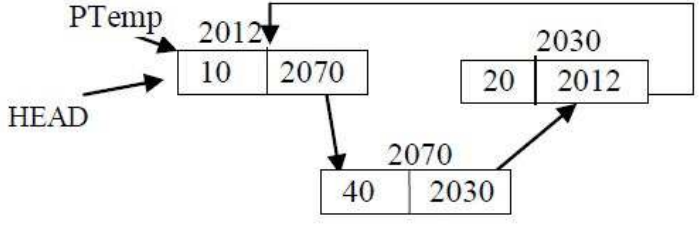
In the first step Temp points to the first node of the circular linked list.



(a)



(b)



(c)

Algorithm to delete a node from the beginning

```
ALGORITHM: CIRDEL (HEAD)
1. [Make Temp to point the first node]
   Set Temp = HEAD
2. [Check for empty list]
   If HEAD = NULL then
      Print: "The linked list is empty"
3. Else
         [Linked list contains single element]
      a) If HEAD = Temp→LINK then
            Set HEAD = NULL
         Else
            P=HEAD
            Repeat while P→LINK ≠ HEAD do
              Set P = P→LINK
            [End of loop]
      b) HEAD=HEAD→LINK
      c) P→LINK=HEAD
4. Set ITEM = Temp→DATA
5. Deallocate memory for Temp
6. Return
```

Doubly Linked List

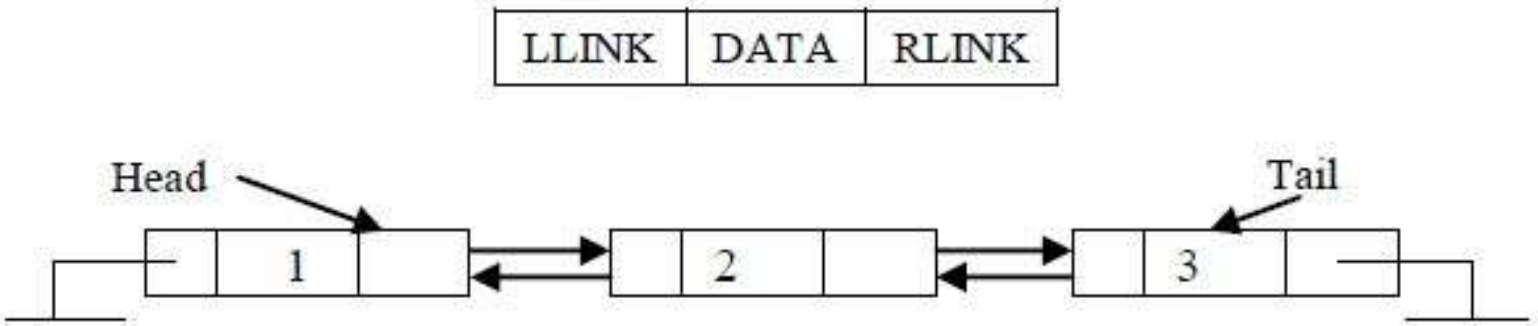
A double linked list is defined as a collection of nodes in which each node has three parts: **data, llink, rlink.**

Data contains the data value for the node, llink contains the address of node before it and rlink contains the address of node after it.

A generic doubly linked list node can be designed as:

```
struct node
{
    int data;
    struct node* rlink;
    struct node* llink;
};
```

Doubly Linked Lists

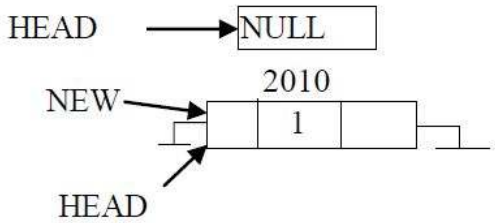


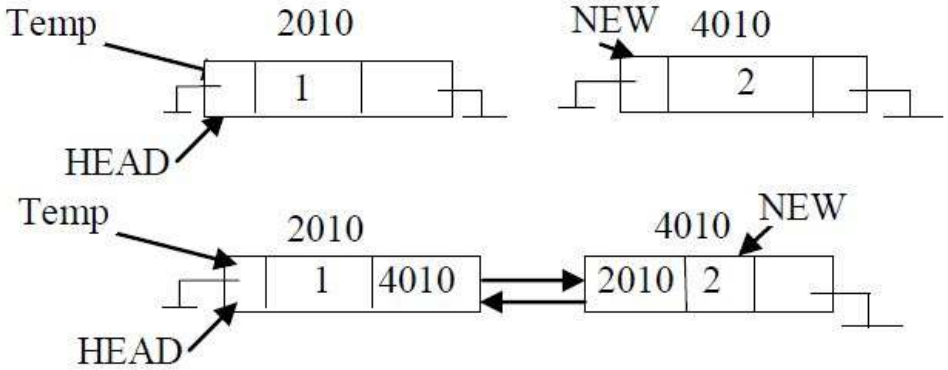
Doubly Linked Lists

Algorithm to create Doubly Linked List

```
Algorithm: DLL_CREATE(HEAD, ITEM)
1. [Create the new node]
   a) Allocate memory for NEW node
   b) IF NEW = NULL then Print: "Memory not Available" and Return
   c) Set NEW→DATA = ITEM
   d) Set NEW→LLINK = NULL
   e) Set NEW→RLINK = NULL
2. [Whether List is empty, head is the content of HEADER]
   If HEAD = NULL then Set HEAD = NEW
3. Else
   a) Set Temp = HEAD
   b) While Temp→RLINK ≠ NULL do
         Set Temp = Temp→RLINK
      [End of while]
   c) Set Temp→RLINK = NEW
   d) Set NEW→LLINK = Temp
  [End of IF]
4. Return
```

Doubly Linked Lists



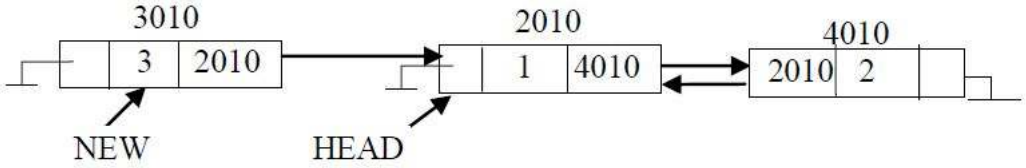


Doubly Linked Lists

Algorithm to insert a node at the beginning

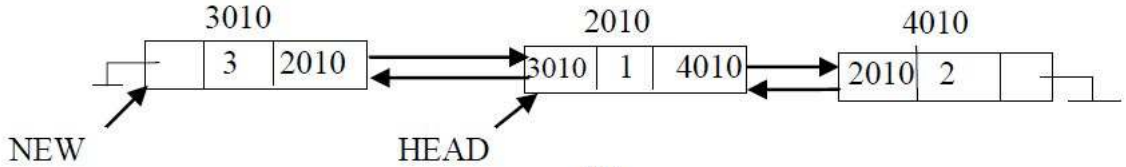
```
Algorithm: DLL_ADD_BEG (HEAD, ITEM)
1. [Create the new node]
   a) Allocate memory for NEW node.
   b) IF NEW = NULL then Print: "Memory not Available"  and Return
   c) Set NEW→DATA = ITEM
   d) Set NEW→RLINK = HEAD
   e) Set NEW→LLINK = NULL
2. Set HEAD→LLINK =NEW
3. [Make the HEADER to point to the NEW node]
   Set HEAD = NEW
4. Return
```

Initially a NEW node is created where rlink of NEW node contains the address of the HEAD node.



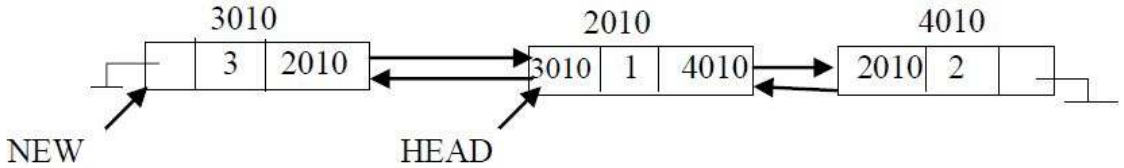
(a)

After that llink of HEAD node is assigned to the address of NEW node.



(b)

At last the HEAD pointer moves to the NEW node which is depicted in the following figure



(c)

Algorithm to insert a node before a node pointer

```
Algorithm: DLL_ADD_BEFORE (HEAD, ITEM, P)
1. Set PRE = P→LLINK
2. [Create the new node]
   a) Allocate memory for NEW node
   b) IF NEW = NULL then Print: Overflow and Return
   c) Set NEW→DATA = ITEM
   d) Set NEW→LLINK = PRE
   e) Set NEW→RLINK = P
3. Set PRE→RLINK = NEW
4. Set P→LLINK = NEW
5. Return
```

Algorithm to insert a node at the end

```
Algorithm: ADD_END (HEAD, ITEM)
1. [Make temp to point to the first node]
   Set Temp = HEAD
2. Repeat while Temp→RLINK ≠ NULL
      Set Temp = Temp →RLINK
   [End of loop]
3. [Create the new node]
   a) Allocate memory for NEW node
   b) IF NEW = NULL then Print: Overflow and Return
   c) Set NEW→DATA = ITEM
   d) Set NEW→RLINK = NULL
   e) Set NEW→LLINK = Temp
4. Set Temp→RLINK = NEW
5. Return
```

Doubly Linked Lists

Algorithm to traverse in forward direction

```
Algorithm: DLL_FTRAVERSE (HEAD)
1. [Check for empty list]
   If HEAD = NULL then
      i) Print: "The linked list is empty"
      ii) Return
2. Set Temp = HEAD
3. Repeat while Temp ≠ NULL
   i) Print: Temp→DATA
   ii) Set Temp = Temp→RLINK
   [End Of Loop]
4. Return
```

Algorithm to traverse in backward direction

```
Algorithm: DLL_BTRAVERSE (HEAD)
1. [Check for empty list]
   If HEAD = NULL then
      i) Print: "The linked list is empty"
      ii) Return
2. [Make temp to point to the first node]
   Set Temp = HEAD
3. Repeat while Temp→RLINK ≠ NULL
      Set Temp = Temp →RLINK
   [End of loop]
4. Repeat while Temp ≠ NULL
      i) Print: Temp→DATA
      ii) Set Temp = Temp →LLINK
   [End of loop]
5. Return
```

Algorithm to delete a node from the beginning

```
Algorithm: DLL_DELETE_BEG (HEAD)
1. [Check for empty list]
   IF HEAD = NULL then
      a) Print: "The linked list is empty"
      b) Return
2. [Make temp to point the first node]
   Set Temp = HEAD
3. Set HEAD = Temp→RLINK
4. Set HEAD→LLINK = NULL
5. Set ITEM = Temp→DATA
6. Set Temp→RLINK = NULL
7. Set Temp→LLINK = NULL
8. Deallocate memory for Temp Node
9. Return
```

Algorithm to delete a node from the end

```
Algorithm: DLL_DELETE_END (HEAD, ITEM)
1. [Check for empty list]
   IF HEAD = NULL then
      a) Print: "The linked list is empty"
      b) Return
2. Repeat step while Temp→RLINK ≠ NULL
      a) Set PTemp = Temp
      b) Set Temp = Temp→RLINK
   [End of loop]
3. Set ITEM = Temp→DATA
4. Set Temp→LLINK = NULL
5. Set PTemp→RLINK = NULL
6. Deallocate memory for Temp Node
7. Return
```

Doubly Linked Lists

Algorithm to delete a node after a given node pointer

```
Algorithm: DLL_DELETE_AFTER (HEAD, ITEM, P)
1. Set Temp = P→RLINK
2. Set POST = Temp→RLINK
3. Set P→RLINK = POST
4. IF POST ≠ NULL then Set POST→LLINK = P
5. Set ITEM = Temp→DATA
6. Set Temp→RLINK = NULL
7. Set Temp→LLINK = NULL
8. Deallocate memory for Temp Node
9. Return
```

Algorithm to delete a node before given node pointer

```
Algorithm: DLL_DELETE_BEFORE (HEAD, ITEM, P)
1. Set Temp = P→LLINK
2. Set PRE= Temp→LLINK
3. IF PRE ≠ NULL then Set PRE→RLINK = P
4. Set P→LLINK = PRE
5. Set ITEM = Temp→DATA
6. Set Temp→RLINK = N ULL
7. Set Temp→LLINK = NULL
8. Deallocate memory for Temp Node
9. Return
```

Doubly Linked Lists

Scenario: Simulate a browser's history feature using a doubly linked list. Each node stores:

- URL
- Date and time of the visit

Questions:

- Add a new URL to the history.
- Navigate back and forward between visited URLs.
- Delete history older than a specific date.

Delete a Node from the Beginning of a Circular Doubly Linked List

Steps

1. **Check if the list is empty**
   - If `head == NULL`,
     → Print "List Empty, Deletion not possible"
     → Return `head`
2. **Check if only one node is present**
   - If `head->next == head`(i.e., next points to itself),
     → Free the node
     → Set `head = NULL`
     → Return `head`

General case (more than one node)

- Let `temp = head`
- Let `last = head->prev`(since it is circular DLL)
- Update links:
  - `head = head->next`
  - `head->prev = last`
  - `last->next = head`
- Free `temp`
- Return updated `head`

Delete a Node from the End of a Circular Doubly Linked List

Steps

1. Check if list is empty

- If `head == NULL`
  → Print "List Empty, Deletion not possible"
  → Return `head`

2. Check if only one node is present

- If `head->next == head`
  → Free the node
  → Set `head = NULL`
  → Return `head`

3. General case (more than one node)

- Let `last = head->prev`(because CDLL has backward pointer)
- Let `second_last = last->prev`
- Update links:
  - `second_last->next = head`
  - `head->prev = second_last`
- Free `last`
- Return updated `head`

Circular Doubly Linked Lists

General algorithm for deleting a node from a circular doubly linked list

Traverse to the node at given position:

a. Set temp = head, count = 1

b. Repeat while count < pos

i. Set temp = temp->next

ii. Increment count

iii. If temp = head before reaching pos

→ Print “Invalid Position”

→ Stop